MeetAI

Mini Project Report

Submitted in partial fulfilment for the requirements for the award of the degree in

INTEGRATED M.Sc. PROGRAMME IN COMPUTER SCIENCE -DATA SCIENCE

Submitted by

JEEVAN JIJO

REG. NO:223142810010

Under the guidance of

Ms. RINCY JOSE

DEPARTMENT OF DATA SCIENCE

NIRMALA COLLEGE MUVATTUPUZHA (AUTONOMOUS)

(Affiliated to M.G. University, Kottayam)



NIRMALA COLLEGE MUVATTUPUZHA (AUTONOMOUS)

(Affiliated to M.G. University, Kottayam)

CERTIFICATE

This is to certify that the project entitled "MeetAI" submitted in partial fulfilment of the requirements for the award of the degree in Integrated M.Sc. Computer Science – Data Science is a bonafide report of the project done by JEEVAN JIJO (Reg.No:223142810010) during the academic year 2026-2027.

Internal Guide **Head of the Department**

Examiner

ACKNOWLEDGEMENT

First and foremost, I thank God Almighty for His unwavering grace and blessings, which have been a constant source of strength and inspiration throughout this project. I am truly grateful for the divine guidance that has made this journey possible.

I would like to express my deep gratitude to our Principal Rev. Dr. JESTIN K KURIAKOSE for giving me the opportunity to learn at this institution. I would like to express my sincere gratitude to my guide, MS. RINCY JOSE, Assistant Professor in the Department of Integrated MSC Computer Science-Data science, Nirmala College Muvattupuzha (Autonomous), for her unwavering support, invaluable guidance, and expert insights throughout the course of this project. Her constructive feedback and constant encouragement were instrumental in refining and shaping the project. I truly appreciate the time, effort, and dedication she invested in ensuring the successful completion of this work. Her mentorship has played a crucial role in my academic and personal growth, and I am deeply thankful for her guidance.

I am also grateful to my institution and faculty members for providing the necessary resources and knowledge that have been instrumental in the development of this project.

A special appreciation goes to my family and friends for their unwavering support, motivation, and belief in my abilities. Their encouragement has been a source of inspiration during challenging times.

Finally, I would like to acknowledge the online resources, datasets, and research papers that provided valuable insights for the successful implementation of the MeetAI System.

Thank you all for your contributions and support.

ABSTRACT

Meetings are an essential part of academic, professional, and organizational activities. However, manually recording meeting minutes, identifying important discussions, tracking decisions, and monitoring assigned tasks can be time-consuming and inefficient. To address these challenges, *MeetAI – Meeting Intelligence System* is an AI-powered application designed to automate meeting analysis and transform audio and video recordings into meaningful, structured information.

The system uses speech-to-text technology to convert meeting recordings into text and applies artificial intelligence techniques to generate concise summaries, identify key discussion points, extract important decisions, and determine action items assigned to participants. Speaker diarization helps distinguish between different speakers, making meeting conversations easier to understand and review. Furthermore, the system incorporates a conversational chatbot that enables users to ask questions about meeting content and retrieve relevant information through Retrieval-Augmented Generation (RAG). Transcript processing, persistent storage using SQLite, and semantic search using FAISS support efficient information retrieval and access to previously processed meeting content.

By combining speech recognition, natural language processing, speaker identification, and AI-powered information retrieval, MeetAI reduces the effort required for manual documentation and improves the accessibility of meeting information. The proposed system provides a practical solution for students, professionals, and organizations seeking to improve meeting productivity, simplify follow-up activities, and ensure that important information is not overlooked

CONTENTS

CHAPTER 1 INTRODUCTION 1

1.1 Need for Study 2

1.1.1 Problem Statement 2

1.2 Objectives 2

1.2.1 Primary Objective 2

1.2.2 Secondary Objective 3

1.3 Summary 3

CHAPTER 2 LITERATURE REVIEW 4

CHAPTER 3 SYSTEM ANALYSIS 7

3.1 Model Architectures 7

3.2 Project Pipeline 9

3.3 Vector Database and RAG Architecture 11

3.4 Feasibility Analysis 12

3.4.1 Technical Feasibility 12

3.4.2 Economic Feasibility 12

3.4.3 Operational Feasibility 13

CHAPTER 4 SYSTEM ENVIORMNET 14

4.1 Software Enviornment 14

4.1 Hardware Enviornment 17

CHAPTER 5 SYSTEM DESIGN 19

5.1 Model Building 19

3.5.1 Model Planning 19

5.2 Training 20

5.3 Model Evaluation 21

5.4 Testing 21

CHAPTER 6 RESULT AND DISCUSSION 23

CHAPTER 7 MODEL DEPLOYMENT 25

7.1 Key Features 25

7.2 User Interface 25

7.3 Performance 25

7.4 Git History 26

CHAPTER 8 CONCLUSION 27

CHAPTER 9 FUTURE SCOPE AND ENHANCEMENT 28

CHAPTER 10 APENDIX 30

7.4 Screenshots 30

CHAPTER 11 REFERENCES 32

CHAPTER 1

INTRODUCTION

Meetings are essential for communication and decision-making, but manually documenting discussions, decisions, and assigned tasks is time-consuming. MeetAI – Meeting Intelligence System is an AI-based application that automates meeting analysis by converting audio and video recordings into text using speech recognition and speaker diarization. It generates meeting summaries, identifies key points, extracts decisions and action items, and provides a conversational chatbot using Retrieval-Augmented Generation (RAG) to answer questions about meeting content. SQLite stores transcript information, while FAISS enables efficient retrieval of relevant text. MeetAI aims to improve productivity, reduce manual effort, and make meeting information easily accessible.

1.1 NEED FOR STUDY

In today's academic and professional environments, meetings generate a large amount of information, including discussions, decisions, and assigned tasks. Manually documenting and organizing this information requires significant time and effort, and important details may be missed or forgotten. Traditional meeting management methods also make it difficult to retrieve specific information from lengthy recordings and track responsibilities effectively. Therefore, there is a need for an intelligent system that can automate meeting documentation and improve information accessibility.

MeetAI addresses these challenges by using artificial intelligence to convert audio and video recordings into text, distinguish between speakers, generate concise summaries, and identify key points, decisions, and action items. Its AI-powered chatbot enables users to ask questions about meeting content and retrieve relevant information without reviewing entire recordings. By combining speech recognition, natural language processing, and intelligent information retrieval, the system reduces manual effort, saves time, and improves the organization of meeting information. Hence, the study focuses on developing an efficient and user-friendly meeting

intelligence system that enhances productivity, supports better decision-making, and simplifies meeting follow-up activities.

1.1.1 PROBLEM STATEMENT

Traditional meeting management relies heavily on manual note-taking, transcription, and preparation of meeting minutes, which are time-consuming and prone to human error. Important discussions, decisions, and assigned tasks may be overlooked, making it difficult for participants to review meeting outcomes and track responsibilities. Furthermore, retrieving specific information from lengthy audio or video recordings requires considerable effort. Existing approaches may lack integrated features for speaker identification, automatic summarization, action item extraction, and interactive information retrieval. Therefore, there is a need for an AI-powered system that can automatically process meeting recordings, generate meaningful summaries, identify key decisions and action items, distinguish between speakers, and provide quick access to meeting information through an intelligent chatbot. MeetAI aims to address these challenges by providing an integrated and efficient solution for automated meeting analysis, documentation, and information retrieval.

1.2 OBJECTIVES

1.2.1 PRIMARY OBJECTIVE

To develop an AI-powered Meeting Intelligence System, MeetAI, that automatically processes audio and video recordings to generate meeting summaries, identify key discussion points, extract decisions and action items, and provide efficient access to meeting information**.**

1.2.2 SECONDARY OBJECTIVES

- **Automated Transcription:** To convert meeting audio and video recordings into text using speech recognition technology.
- **Speaker Diarization:** To identify and distinguish different speakers in meeting conversations.
- Meeting Summarization: To generate concise summaries highlighting the main topics and important discussions.
- **Action Item Extraction:** To automatically identify decisions, assigned tasks, and follow-up activities discussed during meetings.
- **AI-Powered Chatbot:** To enable users to ask questions about meeting content and obtain context-relevant answers.
- **Efficient Information Retrieval:** To use Retrieval-Augmented Generation (RAG) and FAISS-based semantic search to retrieve relevant information from meeting transcripts.
- **Persistent Data Storage:** To store transcript-related information using SQLite, enabling efficient access to previously processed meeting data.
- **Improved Productivity:** To reduce manual documentation efforts, save time, and improve the accessibility and organization of meeting information.

1.3 SUMMARY

MeetAI is an AI-powered Meeting Intelligence System designed to automate meeting analysis and information management. It converts audio and video recordings into text, distinguishes between speakers, generates concise summaries, and extracts key discussion points, decisions, and action items. The system integrates an AI-powered chatbot that enables users to ask questions and retrieve relevant information from meeting transcripts using Retrieval-Augmented Generation (RAG). SQLite provides persistent data storage, while FAISS enables efficient semantic search and information retrieval. By combining speech recognition, natural language processing, speaker diarization, and AI-based question answering, MeetAI reduces manual effort, saves time, and improves information accessibility. Overall, it offers an efficient solution for meeting documentation, knowledge management, and task identification.

CHAPTER 2

LITERATURE REVIEW

2.1.1 Research paper 1

Automatic Meeting Summarization and Topic Detection System

(Data Technologies and Applications, 2018)[1]

Huang, Hsieh, and Wang (2018), in their research paper, proposed a system to automate meeting documentation using speech recognition, topic detection, and text summarization. The system used Latent Dirichlet Allocation (LDA) and TextTiling to identify topics and divide meeting transcripts into meaningful segments. TextTeaser was then used to generate summaries. This research demonstrated the potential of combining speech recognition and text analysis to reduce manual effort in meeting documentation. However, the system relied on traditional summarization techniques, which may not fully capture complex discussions or contextual relationships.

2.1.2 Research paper 2

Abstractive Meeting Summarization: A Survey

(Transactions of the Association for Computational Linguistics, 2023) [2]

Rennard, Shang, Hunter, and Vazirgiannis (2023), in their paper, reviewed existing approaches, datasets, models, and evaluation metrics for meeting summarization. The study highlighted the importance of generating natural-language summaries that capture the essential meaning of lengthy, multi-speaker conversations. It also discussed challenges such as conversational complexity, information selection, and the preservation of important details. This research provides a foundation for developing systems like MeetAI that generate concise and meaningful summaries from meeting transcripts.

2.1.3 Research paper 3

Reconstruct Before Summarize: An Efficient Two-Step Framework for Condensing and Summarizing Meeting Transcripts

(Proceedings of EMNLP, 2023) [3]

Tan et al. (2023), in their research paper, proposed a framework for improving the efficiency of meeting summarization. Their method first identifies essential content through transcript reconstruction and then generates summaries using a summarization model. The approach was evaluated using the AMI and ICSI meeting datasets and demonstrated improvements over previous methods. This study highlights the importance of processing lengthy transcripts efficiently, a challenge also addressed by MeetAI through transcript chunking and organized information retrieval.

2.1.4 Research paper 4

MeetingQA: Extractive Question-Answering on Meeting Transcripts

(Proceedings of the Annual Meeting of the Association for Computational Linguistics, 2023) [4]

Prasad et al. (2023), in their paper, introduced a dataset designed to evaluate question-answering systems on meeting conversations. The study emphasized that useful information may be distributed across multiple statements and speakers, making it challenging to retrieve accurate answers from lengthy transcripts. The researchers found that existing models still faced difficulties in answering meeting-related questions compared with human performance. This research motivates the integration of an AI-powered chatbot in MeetAI, allowing users to ask questions about meeting content instead of manually searching through entire transcripts.

2.1.5 Research paper 5

Summaries, Highlights, and Action Items: Design, Implementation and Evaluation of an LLM-Powered Meeting Recap System

(Proceedings of the ACM on Human-Computer Interaction, 2023) [5]

Asthana, Hilleli, He, and Halfaker (2023), in their study, investigated the use of large language models to generate meeting recaps. The researchers explored two formats: important highlights and structured hierarchical meeting minutes. Their evaluation showed that different recap formats support different user needs, including quickly understanding discussions and reviewing detailed meeting outcomes. This research supports the approach adopted by MeetAI, which aims to organize meeting information into summaries, key points, decisions, and action items.

Research Gap

The reviewed studies demonstrate significant progress in automatic transcription, meeting summarization, action item detection, and question answering. However, these tasks are often studied or implemented as separate components, and challenges remain in integrating them into a single, accessible system. Long meeting transcripts can be difficult to process and search, while identifying speakers, preserving contextual information, and retrieving accurate answers remain important concerns.

MeetAI addresses these challenges by integrating audio and video transcription, speaker diarization, AI-based summarization, action item extraction, and conversational question answering into one application. It also uses SQLite for persistent transcript-related storage and FAISS for semantic similarity search. This integrated approach aims to simplify meeting documentation and make important meeting information easier to access and review.

CHAPTER 3

SYSTEM ANALYSIS

3.1 MODEL ARCHITECTURES

MeetAI uses a combination of deep learning models and natural language processing techniques to automate meeting analysis. The system integrates Faster-Whisper for speech-to-text conversion, PyAnnote for speaker diarization, Sentence Transformers for semantic embeddings, and the Qwen3 language model through Ollama for summarization and information extraction. FAISS is used for efficient similarity-based retrieval of meeting transcript segments. These models work together to generate meeting summaries, identify key discussion points, extract decisions and action items, and support an AI-powered chatbot using Retrieval-Augmented Generation (RAG).

3.1.1 Faster-Whisper

Faster-Whisper is an optimized implementation of OpenAI's Whisper automatic speech recognition model that uses the CTranslate2 inference engine for efficient speech processing. It converts audio from meeting recordings into text while maintaining the accuracy of the original Whisper model. MeetAI uses the Whisper Large V3 Turbo model to transcribe audio extracted from uploaded meeting recordings. The model processes the audio and generates text segments containing the recognized speech and timestamps. These timestamps help associate the transcribed content with specific portions of the original recording. The generated transcript serves as the primary input for subsequent processing, including speaker identification, summarization, decision extraction, and action-item identification.

3.1.2 PyAnnote Speaker Diarization

PyAnnote is a deep learning-based speaker diarization framework used to determine who spoke and when during a meeting. It analyses audio signals to identify changes in speakers and assigns speaker labels to different segments of the recording. MeetAI uses PyAnnote 3.1 to distinguish between participants and organize the meeting conversation according to speaker turns. The diarization output

is aligned with the transcription segments to associate recognized speech with the corresponding speaker and timestamps. This process improves transcript readability and helps the system identify individual contributions, supporting the generation of structured meeting summaries and the analysis of discussions, decisions, and assigned responsibilities.

3.1.3 Qwen3 Language Model

Qwen3 is a large language model used in MeetAI for natural language understanding and generation. The system runs the Qwen3 8B model locally through Ollama, allowing meeting information to be processed without depending on external language-model APIs. The model analyses the transcribed meeting content to generate concise summaries, identify important discussion points, extract decisions, and recognize action items. It can also classify and organize extracted information into structured outputs. In the chatbot module, Qwen3 generates responses based on relevant information retrieved from stored meeting transcripts. This integration enables users to ask questions about previous meetings and receive contextual answers grounded in the available transcript data.

3.1.4 Sentence Transformers

Sentence Transformers is a framework for generating numerical representations, known as embeddings, from text. MeetAI uses the all-MiniLM-L6-v2 model to convert transcript chunks into dense vector representations that capture semantic relationships between sentences and passages. Before embedding generation, the transcript is divided into smaller chunks to support efficient storage and retrieval. Each chunk is transformed into a vector that represents its semantic meaning. These embeddings enable the system to identify transcript passages that are conceptually related to a user's question, even when the wording differs. The generated vectors are stored in the FAISS index and used by the RAG chatbot to retrieve relevant meeting information

3.1.5 FAISS Vector Search

FAISS (Facebook AI Similarity Search) is a vector-search library designed for efficient similarity searching across high-dimensional embeddings. In MeetAI, FAISS is used to retrieve relevant transcript chunks based on the semantic similarity between stored embeddings and a user's query

embedding. When a user submits a question through the chatbot, the query is converted into an embedding using Sentence Transformers. FAISS compares this embedding with the stored transcript vectors and returns the most relevant segments. These retrieved passages are provided as contextual information to the Qwen3 language model, which generates an answer based on the meeting content. This retrieval process forms a core component of the Retrieval-Augmented Generation (RAG) architecture and improves access to information across meeting recordings.

3.2 PROJECT PIPELINE

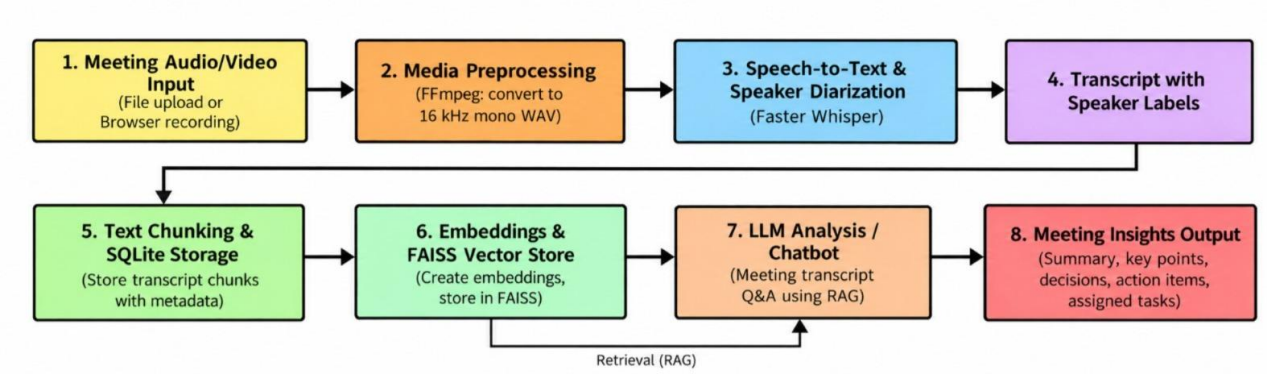


Fig 3.2 Project Pipeline

1. Meeting Audio/Video Input:

The first step of MeetAI is to collect meeting recordings from users. The system allows users to upload audio or video files in formats such as MP3, WAV, M4A, MP4, and WebM. It also supports recording meetings directly through the browser. These recordings serve as the primary input for the meeting analysis process

2. Media Preprocessing:

In this stage, the uploaded audio or video file is processed using FFmpeg. The audio is extracted from video files, converted into a standard format, and resampled to 16 kHz mono WAV audio when required. This preprocessing ensures that the audio is suitable for speech recognition and helps maintain consistency throughout the processing pipeline.

3. Speech-to-Text and Speaker Diarization:

The preprocessed audio is passed to the Faster Whisper speech recognition model, which converts spoken words into text. The system also identifies different speakers using speaker diarization, where supported by the configured pipeline. This helps distinguish who spoke at different points in the meeting and produces a transcript with timestamps and speaker information.

4. Transcript with Speaker Labels:

After transcription, the recognized speech is organized into a readable transcript. Speaker labels and timestamps are associated with the corresponding speech segments wherever available. This makes it easier to understand the conversation, follow discussions, and identify which participants contributed specific information during the meeting.

5. Text Chunking and SQLite Storage:

The generated transcript is divided into smaller text segments, known as chunks, to simplify information retrieval and analysis. Each chunk can contain metadata such as speaker information and timestamps. These chunks are stored persistently in an SQLite database, allowing the system to retrieve transcript information efficiently for later analysis and chatbot queries.

6. Embeddings and FAISS Vector Store:

In this stage, the transcript chunks are converted into numerical representations called embeddings using a sentence-transformer model. These embeddings capture the semantic meaning of the text and are stored in a FAISS vector store. FAISS enables efficient similarity searches, helping the system find transcript sections relevant to a user's question instead of searching the entire transcript manually.

7. LLM Analysis and Chatbot:

The system uses a Large Language Model (LLM) to analyze meeting transcripts and answer user questions. Through Retrieval-Augmented Generation (RAG), the chatbot retrieves relevant

transcript chunks from the FAISS vector store and uses them as context to generate meaningful responses.

8. Meeting Insights Output:

The final stage presents useful information extracted from the meeting. MeetAI generates meeting summaries, key discussion points, important decisions, action items, and assigned tasks. These outputs help users understand the main outcomes of a meeting without reviewing the entire recording. The system improves meeting productivity by organizing information into a structured and easily accessible format.

3.3 VECTOR DATABASE AND RAG ARCHITECTURE.

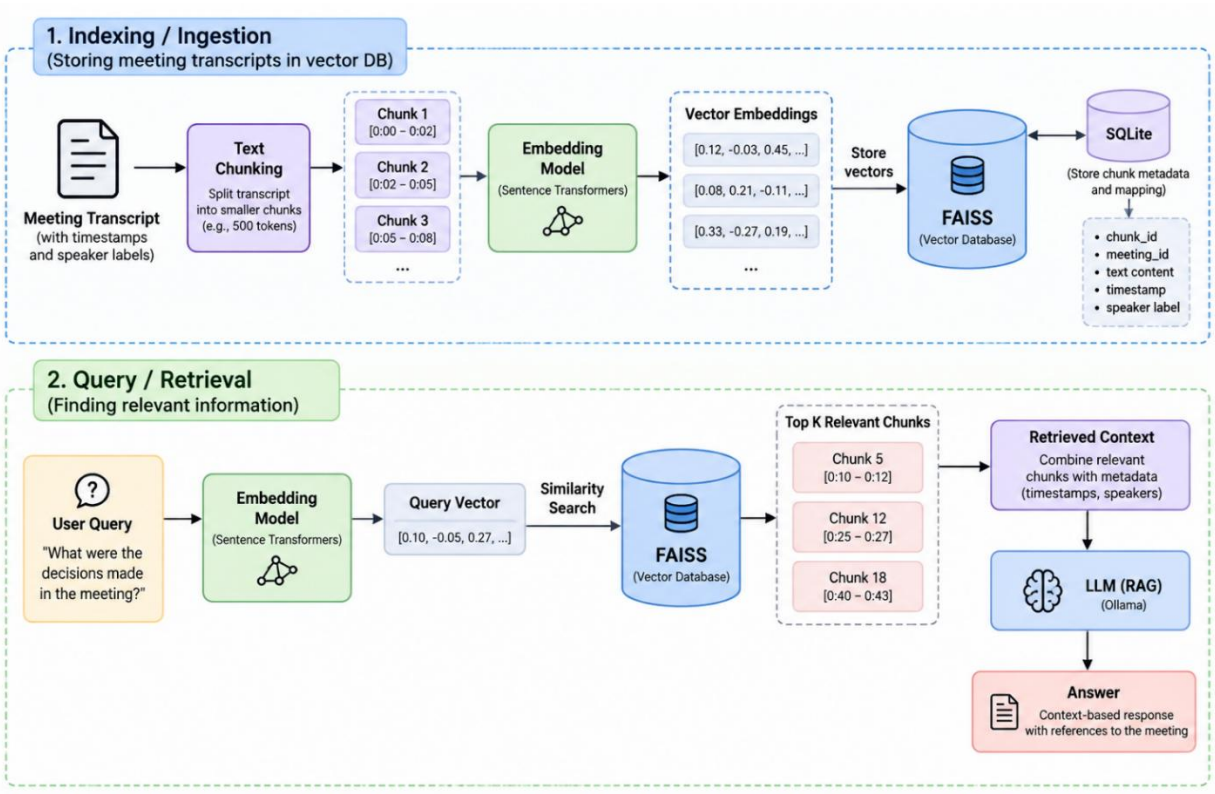


Fig 3.3 Vector DB and RAG Architecture.

3.4 FEASIBILITY ANALYSIS

Feasibility analysis is the process of evaluating whether a proposed system can be developed and operated successfully within the available technical, economic, and operational resources. The feasibility of the MeetAI project is evaluated in terms of technical feasibility, economic feasibility, and operational feasibility.

3.4.1 Technical Feasibility

MeetAI is technically feasible because it uses established technologies, open-source libraries, and pre-trained artificial intelligence models. The system is developed using Python and FastAPI for backend processing, Next.js and React for the frontend, and SQLite for database management. The faster-whisper model is used to convert speech into text, while PyAnnote supports speaker identification. Ollama provides local language models for generating meeting summaries, identifying key points, extracting decisions and action items, and answering questions through Retrieval-Augmented Generation (RAG). FAISS supports efficient semantic searching of meeting transcripts. The system uses FFmpeg for audio and video preprocessing and ReportLab for generating PDF reports. It can operate on Windows 10 or Windows 11 with a suitable processor and at least 16 GB of RAM. An NVIDIA GPU with 8 GB of VRAM is recommended for improved processing speed, although CPU-based processing is also supported. Since the AI models and meeting data are processed locally, the system reduces dependence on external cloud services and helps protect confidential meeting information. Therefore, the availability of suitable technologies and hardware makes MeetAI technically feasible.

3.4.2 Economic Feasibility

MeetAI is economically feasible because it primarily uses open-source software, libraries, and locally executed AI models. Technologies such as Python, FastAPI, Next.js, SQLite, faster-whisper, FAISS, and Ollama can be used without recurring software licence fees for typical project use. This reduces the overall development and operational costs compared with subscription-based meeting intelligence

platforms. The main expenses involve computer hardware, electricity consumption, storage, development time, and system maintenance. Users who already own a compatible computer can implement and operate the system without purchasing additional hardware. A dedicated GPU may improve performance, but CPU-based execution provides an alternative for systems without a suitable GPU. Since meeting recordings and transcripts are stored locally, recurring cloud processing and storage charges can also be avoided. Thus, MeetAI provides a cost-effective solution for students, small teams, and organisations that require automated meeting analysis while maintaining control over their data.

3.4.3 Operational Feasibility

MeetAI is operationally feasible because it provides a user-friendly web interface that simplifies meeting recording, file uploading, transcription, analysis, and information retrieval. Users can upload audio or video recordings, view timestamped transcripts, identify speakers, review meeting summaries, and access important decisions and action items through a single interface.

The system also provides an interactive chatbot that answers questions based on meeting transcripts and supports evidence-based responses with timestamps. Users can edit extracted information, rename speakers, track action items, play recordings alongside transcripts, and export meeting minutes in Markdown, JSON, and PDF formats. Background processing allows lengthy recordings to be analysed without blocking the interface.

The application is designed to run locally, reducing reliance on internet connectivity during normal processing once the required software and models are installed. However, initial setup requires installing the necessary dependencies and configuring the local AI models. Speaker diarization also requires the appropriate Hugging Face access and model permissions; otherwise, the system can continue using its fallback mechanism. Although the current version is designed primarily for a single local user, its features are suitable for managing meeting information efficiently. Therefore, MeetAI is operationally feasible for its intended use.

CHAPTER 4

SYSTEM ENVIRONMENT

The system environment specifies the hardware and software configurations required to develop, execute, and maintain the proposed MeetAI – Meeting Intelligence System. These requirements help developers implement the system effectively and enable users to determine whether their computing environment meets the application's needs. The system environment includes the software tools, programming languages, libraries, frameworks, and hardware components required for meeting transcription, speaker diarization, summarization, and action-item extraction.

4.1 Software Environment

The software environment specifies the development tools, programming languages, frameworks, libraries, and supporting applications used in the implementation of MeetAI.

- **Development Tool:** Visual Studio Code
- **Programming Language:** Python
- **Operating System:** Windows 10/11
- **Frontend:** Next.js, React, and TypeScript
- **Backend:** FastAPI
- **Speech Recognition:** faster-whisper (CTranslate2)
- **Speaker Diarization:** PyAnnote Audio
- **Language Model Integration:** Ollama
- **Database:** SQLite

- **Vector Search:** FAISS CPU
- **Text Embeddings:** Sentence Transformers
- **Audio and Video Processing:** FFmpeg
- **PDF Generation:** ReportLab
- **Version Control:** Git and GitHub
- **Web Browser:** Google Chrome or Microsoft Edge
- **Internet Connection:** Required for initial model downloads and optional external services; normal AI processing can run locally after setup.

Description of Software Components

Python: Python is the primary programming language used for backend development, audio processing, database operations, transcript analysis, and integration of artificial intelligence components.

faster-whisper: faster-whisper is the speech recognition component used to convert meeting audio into text. It is based on the Whisper model and uses the CTranslate2 inference engine for efficient processing. It supports timestamped transcription and can operate using a compatible GPU or CPU.

PyAnnote Audio: PyAnnote Audio is used for speaker diarization, which identifies different speakers and determines when each speaker is talking. The resulting speaker information is aligned with the transcript to make meeting discussions easier to understand. Access to the required pretrained model may require a Hugging Face account, authorization, and acceptance of its usage conditions.

Ollama: Ollama runs language models locally to analyse meeting transcripts, generate summaries, identify important discussion points, and extract decisions and action items. It also supports the chatbot's question-answering functionality.

SQLite: SQLite is used as the persistent database for storing meeting details, transcripts, speaker information, summaries, decisions, action items, and other meeting-related data. It provides a lightweight storage solution without requiring a separate database server.

FAISS (Facebook AI Similarity Search): FAISS manages vector indexes and supports similarity-based retrieval of transcript segments. It helps the chatbot retrieve relevant meeting information when users ask questions about recorded discussions.

Sentence Transformers: Sentence Transformers converts transcript chunks into numerical vector representations called embeddings. These embeddings allow FAISS to retrieve semantically relevant information from meeting transcripts.

FFmpeg: FFmpeg processes audio and video files and converts audio into a standardized format suitable for transcription. It supports operations such as extracting audio, converting sample rates, and converting stereo audio to mono when required.

Web Technologies: Next.js, React, and TypeScript provide the frontend interface, allowing users to upload recordings, view transcripts and summaries, review action items, and interact with the chatbot. FastAPI handles backend requests and coordinates the processing pipeline.

Git and GitHub: Git tracks changes to the source code, while GitHub supports repository hosting, version management, and collaborative development.

Visual Studio Code: Visual Studio Code is the primary development environment used to write, edit, debug, and maintain the application's source code.

Web Browser: A modern browser, such as Google Chrome or Microsoft Edge, provides access to the MeetAI web interface and its features.

Internet Connection: An internet connection may be required to download pretrained models and dependencies during installation. Once the necessary models and software are available locally, transcription, diarization, transcript analysis, and chatbot operations can run locally without sending meeting recordings to an external transcription service.

4.2 HARDWARE ENVIRONMENT

The hardware environment specifies the minimum and recommended computer configurations required to execute the MeetAI system. Since the system uses cloud-based speech recognition and speaker diarization, a dedicated GPU is not essential for the primary transcription pipeline.

Minimum Configuration:

- **Processor:** Intel Core i3 or equivalent.
- **RAM:** 4 GB.
- **Storage:** 10 GB of available disk space.
- **GPU:** Not required for the primary cloud-based transcription process.
- **Internet:** Internet connection required for cloud API access.
- **Display:** 1366 × 768 resolution or higher.

Recommended Configuration:

- **Processor:** Intel Core i5 or equivalent, with four or more CPU cores.
- **RAM:** 8 GB or higher.
- **Storage:** 20 GB or more of available disk space.
- **GPU:** Integrated graphics are sufficient for normal operation.
- **Internet:** Stable broadband connection.
- **Display:** Full HD resolution (1920 × 1080) or higher.

Development Environment:

- **Processor:** Intel Core i5 or equivalent processor.
- **RAM:** 8 GB or higher.
- **Storage:** SSD with sufficient free space for project files, recordings, database files, and vector indexes.
- **GPU:** Not required for cloud-based speech recognition and speaker diarization.

- **Internet:** Stable broadband connection for API communication and media uploads.
- **Operating System:** Windows 10/11 or a compatible Linux distribution.

The actual hardware requirements depend on the size and duration of meeting recordings, the number of concurrent operations, and the additional language models or services used by the application. Systems with higher RAM and faster storage may provide better performance when processing large recordings or managing multiple meetings.

CHAPTER 5

SYSTEM DESIGN

System design describes the architecture and workflow of MeetAI – Meeting Intelligence System. The system processes meeting audio and video recordings to generate transcripts, identify speakers, summarize discussions, extract key points, decisions, and action items, and answer user questions about meetings.

The system uses faster-whisper for speech recognition and PyAnnote Audio for speaker diarization. FFmpeg is used to process audio and video files when required. The generated transcript is divided into smaller chunks and stored in SQLite. Sentence Transformers converts the text chunks into numerical embeddings, while FAISS performs semantic search to retrieve relevant information. The chatbot uses Retrieval-Augmented Generation (RAG) with a locally running language model through Ollama to answer user questions based on the retrieved meeting context.

The modular architecture separates transcription, speaker identification, data storage, semantic retrieval, and chatbot processing into distinct components. This makes MeetAI easier to maintain, test, and improve while supporting efficient and locally processed meeting analysis.

5.1. Model Building

5.1.1. Model Planning

The model planning phase of MeetAI focuses on selecting and integrating suitable pretrained artificial intelligence models to automate meeting transcription, speaker identification, information extraction, and intelligent question answering. The system follows a local-first architecture, ensuring that audio recordings, transcripts, and meeting information are processed locally rather than sent to external cloud services. The selected models and techniques are designed to balance transcription accuracy, processing speed, computational requirements, and data privacy.

The system uses faster-whisper for automatic speech recognition (ASR), converting meeting audio into text with word-level timestamps. PyAnnote Audio 3.1 is used for speaker diarization to identify different speakers and determine when each person speaks. The transcribed text is aligned with speaker information and divided into smaller semantic chunks for efficient processing. A locally hosted large language model through Ollama, such as Llama 3.2 or Qwen, analyzes the transcript to generate meeting summaries, key discussion points, decisions, action items, important dates, and unresolved questions.

For intelligent information retrieval, MeetAI uses the Sentence Transformers all-MiniLM-L6-v2 embedding model to convert transcript chunks into numerical vectors. These vectors are stored and searched using FAISS, enabling the Retrieval-Augmented Generation (RAG) chatbot to retrieve relevant meeting information and answer user questions with timestamp-based citations. A grounding verification module checks extracted information against the original transcript to reduce hallucinations and prevent unsupported decisions, assignments, or deadlines.

The system does not involve training or fine-tuning new machine learning models. Instead, it uses pretrained models and inference techniques integrated into a unified processing pipeline. Model execution is planned sequentially to manage GPU memory consumption, with CPU-based fallbacks where supported. This approach enables MeetAI to process meeting recordings efficiently while maintaining local data privacy, reliable information extraction, and evidence-based question answering.

5.2 Training

The MeetAI – Meeting Intelligence System uses pretrained artificial intelligence and machine learning models to process meeting recordings and extract meaningful information. The system does not involve training a new model or fine-tuning existing models. Instead, it integrates pretrained models and uses inference techniques to perform speech recognition, speaker diarization, text analysis, and intelligent information retrieval.

The **faster-whisper** model is used to convert meeting audio into text with word-level timestamps. The pretrained PyAnnote Audio model identifies different speakers and their speaking intervals. A locally hosted large language model, accessed through Ollama, analyzes the generated transcript to

extract summaries, key discussion points, decisions, action items, and important dates. These models are used directly for inference without updating their pretrained weights.

For the chatbot functionality, the **all-MiniLM-L6-v2** embedding model converts transcript chunks into numerical vector representations. FAISS stores and searches these vectors to retrieve relevant information from meeting transcripts. The retrieved information is provided to the local language model to generate context-based answers with timestamp citations. A grounding verification mechanism checks extracted information against the original transcript to reduce unsupported statements and hallucinations.

The system's implementation focuses on model integration, inference configuration, preprocessing, and efficient resource management rather than model training. Models are executed sequentially to reduce GPU memory consumption, with CPU fallback options where supported. This approach enables MeetAI to perform automated meeting analysis while maintaining local data processing and minimizing dependence on external cloud services.

5.2 Model Evaluation

The system can be evaluated by checking transcription accuracy, speaker identification, summary quality, action-item extraction, and retrieval relevance. Improvements can be made by adjusting preprocessing, chunk sizes, retrieval settings, and response-generation instructions.

Since the system primarily integrates pretrained models, these activities represent model integration, processing, and evaluation rather than training a new model from scratch.

4.3 Testing

System testing verifies that MeetAI operates correctly and meets its functional requirements. Testing covers individual modules, interactions between components, and the complete application workflow.

The main testing activities include:

File Upload Testing: Verifies that supported recordings are accepted.

Transcription Testing: Checks whether speech is converted into text correctly.

Speaker Diarization Testing: Evaluates speaker-label accuracy.

Database Testing: Verifies that transcript information is stored and retrieved correctly.

Vector Search Testing: Checks whether relevant transcript chunks are retrieved using FAISS.

Summarization Testing: Evaluates the accuracy and relevance of generated summaries.

Action-Item Testing: Checks whether tasks and decisions are extracted correctly.

Chatbot Testing: Verifies that answers are relevant to the meeting transcript.

Error Handling Testing: Checks how the system handles invalid files, API failures, and retrieval errors.

CHAPTER 6

RESULTS AND DISCUSSION

The MeetAI – Meeting Intelligence System was developed to automate the analysis of meeting recordings and transform spoken conversations into structured, meaningful information. The system processes uploaded audio and video files by extracting and normalizing audio using FFmpeg, converting speech into text using the faster-whisper model, and identifying individual speakers through PyAnnote speaker diarization. The generated transcript includes timestamps and speaker labels, allowing users to review specific discussions without listening to the entire recording. The system also stores meeting information in an SQLite database and uses FAISS vector indexing to support efficient retrieval of relevant transcript content.

The system provides automated meeting summarization and information extraction by identifying key discussion points, decisions, action items, important dates, and unresolved questions from the transcript. The locally hosted language model, accessed through Ollama, analyzes the transcribed content to generate structured meeting information. A grounding verification mechanism checks the extracted information against the original transcript to reduce unsupported statements and improve the reliability of the generated results. In addition, the Retrieval-Augmented Generation (RAG) chatbot enables users to ask questions about meeting discussions and receive context-based answers with timestamp references. The system also supports synchronized audio and video playback, speaker renaming, and exporting meeting minutes in Markdown, JSON, and PDF formats.

Overall, the results demonstrate that MeetAI provides an integrated solution for meeting documentation, analysis, and information retrieval. By combining speech recognition, speaker diarization, natural language processing, vector-based search, and locally executed language models, the system converts unstructured meeting recordings into organized and searchable information. This reduces the effort required for manual note-taking and helps users identify important decisions and assigned tasks more efficiently. However, the system's performance may be affected by background noise, unclear speech, overlapping conversations, and hardware limitations. Speaker identification also depends on the availability and successful execution of the diarization model. Future

improvements can focus on enhancing transcription accuracy, supporting automatic multilingual detection, improving the reliability of information extraction, and introducing real-time transcription during live meetings.

CHAPTER 7

MODEL DEPLOYMENT

MeetAI is deployed as a web application using Next.js 14 and FastAPI. It uses faster-whisper for speech recognition, PyAnnote for speaker diarization, and Ollama for meeting summarization and information extraction. SQLite stores meeting data, while FAISS enables semantic search. The system uses pretrained models for local processing without additional training.

7.1.1 KEY FEATURES

- Audio and video upload.
- Automatic speech-to-text conversion.
- Speaker identification and diarization.
- Meeting summaries and key point extraction.
- Decision and action item extraction.
- RAG-based chatbot for meeting queries.
- Transcript storage and semantic search.
- Export reports in Markdown, JSON, and PDF.

7.1.2 User Interface: The dashboard consists of:

- **Meeting Upload:** Upload audio and video recordings.
- **Transcript Viewer:** Display transcripts with timestamps and speaker labels.
- **Meeting Summary:** View key points, decisions, and action items.
- **AI Chatbot:** Ask questions about meeting content.
- **Media Player:** Play recordings and review discussions.
- **Export Options:** Download meeting reports in Markdown, JSON, and PDF formats.

7.1.3 Performance

MeetAI efficiently processes meeting recordings using faster-whisper for transcription, PyAnnote for speaker diarization, and Ollama for summarization. SQLite stores meeting data, while FAISS enables efficient semantic search. Performance depends on recording quality, file size, available system resources, and model execution time.

7.2 GIT HISTORY

https://github.com/Jeevan-Jijo/MeetAI



Fig 7.1

CHAPTER 8

CONCLUSION

MeetAI – Meeting Intelligence System is designed to automate meeting documentation, transcription, summarization, and information retrieval. Traditional meeting management requires considerable time to review recordings, prepare notes, identify decisions, and track assigned tasks. MeetAI addresses these challenges by processing audio and video recordings and converting conversations into structured, searchable information. Using faster-whisper for speech recognition and PyAnnote for speaker diarization, the system generates transcripts with speaker identification, helping users review discussions without repeatedly listening to lengthy recordings.

The system provides several key features, including transcript generation, speaker identification, meeting summarization, key point extraction, decision identification, and action item extraction. An AI-powered chatbot uses Retrieval-Augmented Generation (RAG) to answer questions based on meeting transcripts. SQLite stores meeting data, while FAISS enables efficient similarity-based information retrieval. These features reduce manual note-taking and help users access important meeting information more efficiently. The system is suitable for educational institutions, corporate organizations, and project teams that rely on meetings for communication and decision-making.

In conclusion, MeetAI demonstrates how artificial intelligence can improve meeting management by transforming recorded conversations into organized and accessible information. It helps save time, improves information retrieval, and simplifies the identification of important decisions and tasks. Future enhancements may include real-time transcription, multilingual support, improved speaker identification, automatic task assignment, calendar integration, and enhanced security. These improvements could make MeetAI a more comprehensive meeting intelligence platform that supports collaboration and productivity.

CHAPTER 9

FUTURE SCOPE AND ENHANCEMENTS

The MeetAI – Meeting Intelligence System has significant potential for future development through the integration of advanced artificial intelligence and natural language processing technologies. One of the major enhancements would be the introduction of real-time meeting transcription, allowing the system to capture conversations and generate transcripts while a meeting is in progress. The system could also support multiple languages, enabling users to transcribe and analyze meetings conducted in different languages. Improved speech recognition and speaker diarization techniques could enhance transcription accuracy, particularly in situations involving background noise, different accents, and overlapping conversations. Furthermore, advanced summarization models could generate concise summaries, highlight important discussion points, and organize meeting content into structured sections, making the system more useful for both short discussions and lengthy business meetings.

Another important area for future development is the enhancement of task management and intelligent information retrieval. The system could automatically identify action items, assign responsibilities to participants, extract deadlines, and track the progress of tasks discussed during meetings. Integration with productivity tools, calendar applications, email services, and project management platforms could help users manage their responsibilities more efficiently. The chatbot could also be enhanced to provide more accurate, context-aware responses and support follow-up questions based on previous discussions. Additionally, the system could generate downloadable meeting reports in formats such as PDF and DOCX, including summaries, key decisions, speaker details, and assigned tasks. These improvements would transform MeetAI from a meeting transcription and summarization tool into a comprehensive platform for meeting analysis, collaboration, and task coordination.

In the future, MeetAI could be developed into a scalable, cloud-based application accessible through web browsers and mobile devices. Cloud integration would enable users to securely store meeting recordings, access transcripts from different devices, and collaborate with team members remotely.

Advanced analytics could also be introduced to identify recurring topics, evaluate meeting participation, track decision-making patterns, and generate insights from multiple meetings over time. To support organizational adoption, the system could incorporate stronger authentication mechanisms, role-based access control, data encryption, and improved privacy management to protect confidential meeting information. Further improvements in processing speed, system scalability, and retrieval accuracy could make the application suitable for educational institutions, corporate organizations, healthcare administration, and other professional environments. With these enhancements, MeetAI could evolve into an intelligent meeting management solution that improves communication, supports informed decision-making, and increases overall productivity.

CHAPTER 10

APPENDIX

10.1 SCREENSHOTS

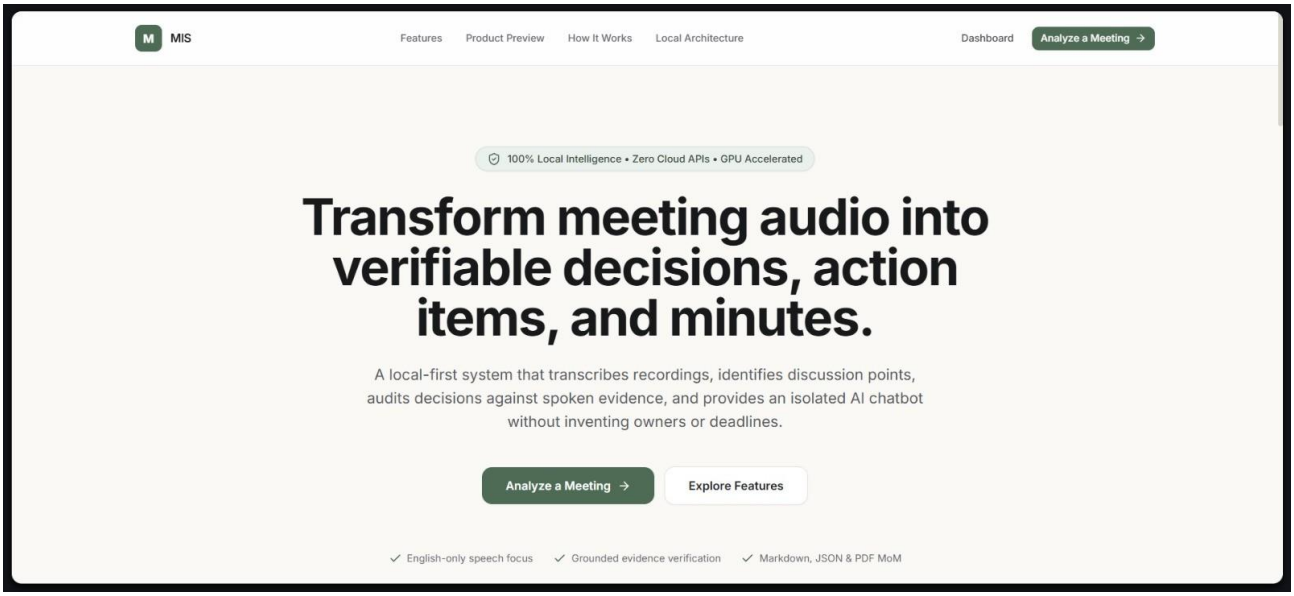


Fig 10.1

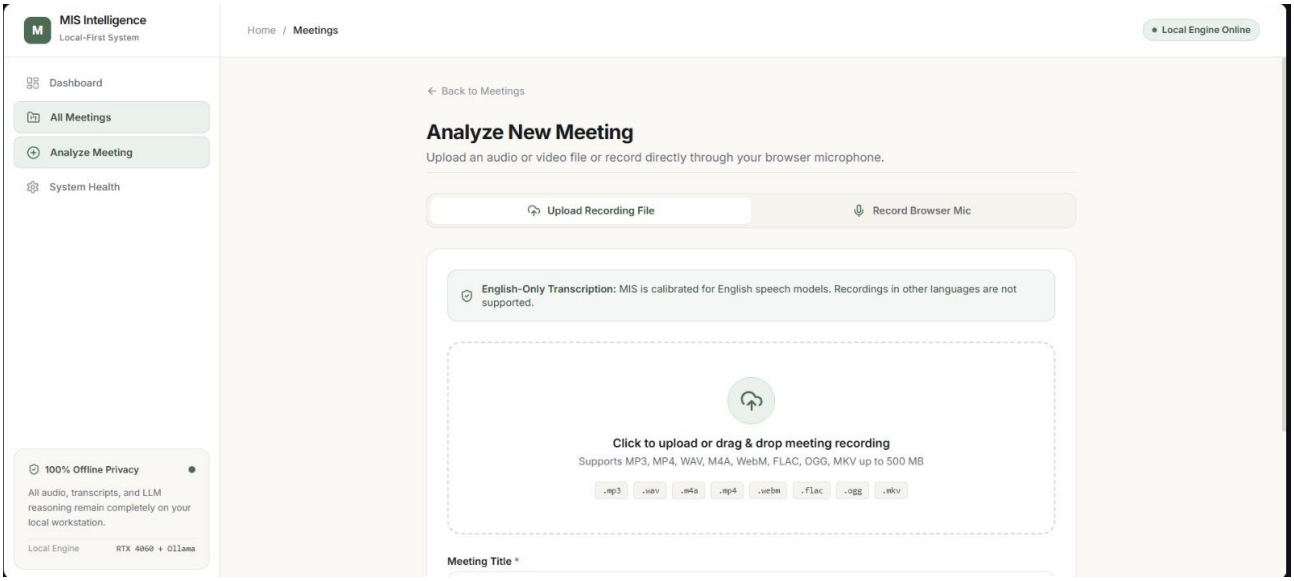


Fig 10.2

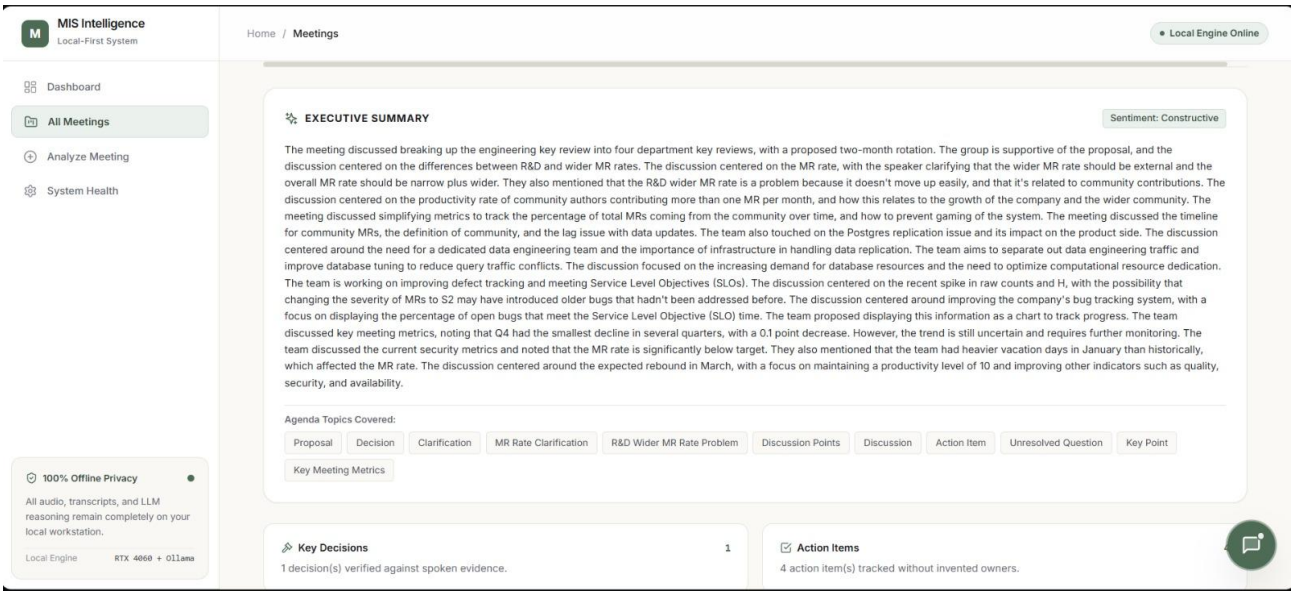


Fig 10.3

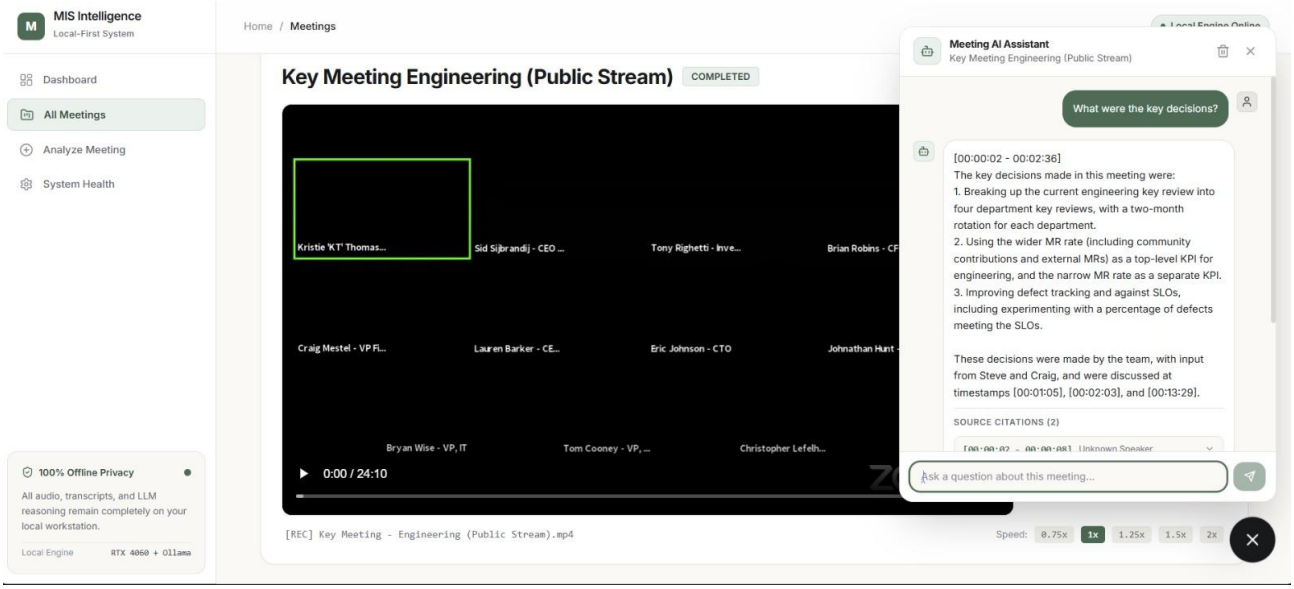


Fig 10.4

CHAPTER 11

REFERENCES

[1] Abstractive Meeting Summarization: A Survey Rennard, V., Shang, G., Hunter, J., & Vazirgiannis, M. (2023). “Abstractive Meeting Summarization: A Survey.” Transactions of the Association for Computational Linguistics, 11, 861–884.

[2] Automatic Meeting Summarization and Topic Detection System Huang, T.-C., Hsieh, C.-H., & Wang, H.-C. (2018). “Automatic Meeting Summarization and Topic Detection System.” Data Technologies and Applications, 52(3), 351–365. DOI: 10.1108/DTA-09-2017-0062.

[3] End-to-End Abstractive Summarization for Meetings: Zhu, C., Xu, R., Zeng, M., & Huang, X. (2020). “End-to-End Abstractive Summarization for Meetings.” arXiv preprint arXiv:2004.02016.

[4] Development of Supervised Speaker Diarization System Based on the PyAnnote Audio Processing Library Khoma, V., Khoma, Y., Brydinskyi, V., & Konovalov, A. (2023). “Development of Supervised Speaker Diarization System Based on the PyAnnote Audio Processing Library.” Sensors, 23(4), 2082. DOI: 10.3390/s23042082.

[5] pyannote.audio 2.1 Speaker Diarization Pipeline: Principle, Benchmark, and RecipeBredin, H. (2023). “pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe.” Proceedings of Interspeech 2023.

[6] Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks Reimers, N., & Gurevych, I. (2019). “Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks.” Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, 3982–3992.

[7] Billion-Scale Similarity Search with GPUs Johnson, J., Douze, M., & Jégou, (2019). “Billion-Scale Similarity Search with GPUs.” IEEE Transactions on Big Data, 7(3), 535–547. DOI: 10.1109/TBDATA.2019.2921572.

[8] Tell Me What I Need to Know: Exploring LLM-Based (Personalized) Abstractive Multi-Source Meeting Summarization Kirstein, F., Ruas, T., Kratel, R., & Gipp, B. (2024). “Tell Me What I Need to Know: Exploring LLM-Based (Personalized) Abstractive Multi-Source Meeting Summarization.” Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track, 920–939.

[9] Dialogue Discourse-Aware Graph Model and Data Augmentation for Meeting Summarization Fe ng, X., Feng, X., Qin, B., & Geng, X. (2021). “Dialogue Discourse-Aware Graph Model and Data Augmentation for Meeting Summarization.” Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI), 3808–3814.

[10] What's Under the Hood: Investigating Automatic Metrics on Meeting Summarization Kirstein, F., Wahle, J. P., Ruas, T., & Gipp, B. (2024). “What's Under the Hood: Investigating Automatic Metrics on Meeting Summarization.” Findings of the Association for Computational Linguistics: EMNLP 2024.